

B.TECH. COMPUTER SCIENCE AND ENGINEERING (AI& ML)
Deenbandhu Chhotu Ram University of Science & Technology Murthal, Sonapat, Haryana
Semester – III (Effective from Session 2025-26)

Course Code: PCCSE201	Minor	TCA	TES	PCA	PES	Total
Title of the Course: DATA STRUCTRES	15	10	50	15	10	100
L-T-P: 3-0-2						
Credits: 4	Durationof Exam: 3 Hours					
Brief Write-Up About the Course (100-150 words) This course provides an in-depth understanding of data structures and their role in efficient algorithm design and software development. Students will learn to organize, manage, and store data using linear structures like arrays, linked lists, stacks, and queues, as well as non-linear structures such as trees and graphs. Emphasis is placed on the application of these structures in solving computational problems, including expression evaluation, tree traversal, graph algorithms, and memory optimization. Additionally, the course covers essential searching and sorting techniques, hashing mechanisms, and collision resolution strategies. Through hands-on programming exercises and conceptual discussions, students will develop problem-solving abilities and algorithmic thinking, laying a strong foundation for advanced topics in computer science.						
Course Objectives: 1. To introduce the fundamental concepts of data structures, their classifications, operations, and associated algorithmic complexities. 2. To enable students to implement and apply linear data structures such as arrays, linked lists, stacks, and queues in solving real-world problems. 3. To develop the ability to construct and manipulate hierarchical and non-linear data structures like trees and graphs, and apply related algorithms effectively. 4. To equip students with the skills to perform efficient searching, sorting, and hashing techniques for data organization and retrieval.						
Unit I Introduction to Data Structures: Classification of Data Structures, Data Structure Operations Complexity of Algorithms, Abstract Data Types (ADT) Arrays: Applications & Advantages, Representation of Linear Arrays in Memory, Insertion & Deletion, Types: One-dimensional, two-dimensional, & multidimensional arrays, Sparse Matrix. Linked Lists: Representation of linked lists in memory, Traversing, Searching, Insertion into, Deletion from linked list, Polynomial Addition, Doubly linked list, Applications of linked list.						
Unit II Stacks and Queues: Operations on Stacks- Push, Pop, Peep, Representation of stacks. Application of stacks, conversion of infix expression to postfix expression, evaluation of postfix expression, Recursion, Representation of Queues, Operations on queues, Priority Queues, Dequeues, Circular Queue, Operations and Applications.						
Unit III Trees: Basic Terminology and Applications, Binary Trees and their Representation, Expression Evaluation, Complete Binary Trees (CBT), Extended Binary Trees, Traversing Binary Trees, Searching, Insertion and Deletion in Binary Search Trees, AVL trees, Threaded trees, Indexing: B Trees, B+ Trees. Graphs: Terminology, Representations and applications, Graphs & Multigraphs, Directed Graphs, Graph Representations (Adjacency Matrix, Adjacency List), Graph Traversals - BFS, DFS, Minimum Spanning trees (Prim’s and Kruskal’s), Shortest path.						

Unit IV

Searching, Sorting methodologies: Array- Linear Search, Binary Search, Bubble sort, Selection Sort, Insertion Sort, Quick Sort, Merge Sort, Radix Sort, Heap Sort.

Hashing: Introduction, Hash functions, collision resolution techniques.

Text Books:

1. Fundamentals of Data Structure by Ellis Horowitz & Sartaj Sahni, Pub, 1983. AW
2. Data Structures with C by Seymour Lipschutz, Schaum's outline by TMH.

Reference Books:

1. An introduction to data structures and application by Jean Paul Tremblay & Pal G. Sorenson (McGraw Hill)
2. Data Structures and Algorithms by A. V. Aho, J. E. Hopcroft and T. D. Ullman, Original edition, Addison-Wesley, 1999, Low Price Edition. Data Structure and Program design in C by Robert Kruse, PHI
3. Introduction to Computer Science- An algorithms approach, Jean Paul Tremblay, Richard B. Bunt, 2002, TMH.
4. Data Structure and Standard Template Library- Willam J. Collins, 2003, T.M.H

Course Outcomes:

Upon successful completion of the course, students will demonstrate the ability to:

1. Identify and explain the appropriate data structure for a given problem and analyze its complexity.
2. Implement linear data structures (arrays, linked lists, stacks, queues) and demonstrate their applications in expression evaluation and memory management.
3. Design and develop tree and graph-based structures for hierarchical data representation and apply traversal, search, and optimization algorithms.
4. Apply efficient searching, sorting, and hashing techniques for problem-solving in large-scale data processing scenarios.

List of Programs:

1. Implement stack as an ADT using Arrays.
2. Implement stack as an ADT using the Linked List ADT.
3. Write a program to evaluate a prefix/postfix expression using stacks.
4. Implement Queue as an ADT using the circular Arrays.
5. Implement Queue as an ADT using the Circular Linked List ADT..
6. Implement singly linked list as an ADT that supports the following operations:
 - (i) Insert an element x at the beginning of the singly linked list.
 - (ii) Insert an element x at ith position in the singly linked list.
 - (iii) Remove an element from the beginning of the singly linked list.
 - (iv) Remove an element from ith position in the singly link.
 - (v) Search for an element x in the singly linked list and return its pointer.
 - (vi) Concatenate two singly linked lists.
7. Implement doubly linked list as an ADT that supports the following operations:
 - (i) Insert an element x at the beginning of the doubly linked list.
 - (ii) Insert an element x at ith position in the doubly linked list.
 - (iii) Insert an element x at the end of the doubly linked list.
 - (iv) Remove an element from the beginning of the doubly linked list.
 - (v) Remove an element from ith position in the doubly linked list.
 - (vi) Remove an element from the end of the doubly linked list.
 - (vii) Search for an element x in the doubly linked list and return its pointer.
 - (viii) Concatenate two doubly linked lists.
8. Implement circular linked list as an ADT which supports the following operations:
 - (i) Insert an element x at the front of the circularly linked list.

- (ii) Insert an element x after an element y in the circularly linked list.
 - (iii) Insert an element x at the back of the circularly linked list.
 - (iv) Remove an element from the back of the circularly linked list.
 - (v) Remove an element from the front of the circularly linked list.
 - (vi) Remove the element x from the circularly linked list.
 - (vii) Search for an element x in the circularly linked list and return its pointer.
 - (viii) Concatenate two circularly linked lists
9. Implement Binary Search Tree as an ADT which supports the following operations:
- (i) Insert an element x.
 - (ii) Delete an element x.
 - (iii) Search for an element x in the BST and change its value to y and then place the node with value y at its appropriate position in the BST.
 - (iv) Display the elements of the BST in preorder, inorder, and postorder traversal.
 - (v) Display the elements of the BST in level-by-level traversal.
 - (vi) Display the height of the BST 10.
10. Implement a Graph Traversals.
11. Implementation of Binary Search Trees.
12. Implementation of AVL Trees.
13. Implementation of Linear Search and Binary Search.
14. Implementation of Insertion Sort and Selection Sort.
15. Implementation of Merge Sort.